A PROJECT REPORT ON

DESIGN AND IMPLEMENTATION OF A CLOUD–EDGE COLLABORATIVE MAPPING AND CONTROL SYSTEM FOR VIRTUAL MOBILE ROBOTS

SUBMITTED TO THE SAVITRIBAI PHULE PUNE UNIVERSITY, PUNE
IN PARTIAL FULFILLMENT OF THE REQUIREMENTS
FOR THE AWARD OF THE DEGREE OF

BACHELOR OF ENGINEERING
(COMPUTER ENGINEERING)

SUBMITTED BY

Student Name: Vivek Gunvant Patil	**Exam No:**
Student Name: Tejas Rajendra Badgujar	**Exam No:**
Student Name: Sharvil Vijayanand Lahute	**Exam No:**
Student Name: Harshal Bharat Bagul	**Exam No:**

UNDER THE GUIDANCE OF
Prof. Varsha Ohol

SANDIP FOUNDATION

DEPARTMENT OF COMPUTER ENGINEERING
Sandip Institute of Technology and Research Centre, Nashik
Nashik - 422213

SAVITRIBAI PHULE PUNE UNIVERSITY
2025 – 2026

SANDIP FOUNDATION

CERTIFICATE

This is to certify that the project report entitled

"DESIGN AND IMPLEMENTATION OF A CLOUD–EDGE COLLABORATIVE MAPPING AND CONTROL SYSTEM FOR VIRTUAL MOBILE ROBOTS"

Submitted by

Student Name: Vivek Gunvant Patil	**Exam No:**
Student Name: Tejas Rajendra Badgujar	**Exam No:**
Student Name: Sharvil Vijayanand Lahute	**Exam No:**
Student Name: Harshal Bharat Bagul	**Exam No:**

are bonafide students of this institute and the work has been carried out by them under the supervision of **Prof. Varsha Ohol**. It is approved for the partial fulfillment of the requirement of Savitribai Phule Pune University, for the award of the degree of **Bachelor of Engineering** (Computer Engineering).

Guide Prof. Varsha Ohol Department of Computer Engineering	Head Dr. Ankita V. Karale Department of Computer Engineering
Principal Dr. Anmol D. Potgantwar	Project External

DEPARTMENT OF COMPUTER ENGINEERING
Sandip Institute of Technology and Research Centre, Nashik
Nashik - 422213
SAVITRIBAI PHULE PUNE UNIVERSITY
2025 – 2026

ACKNOWLEDGEMENT

It gives us immense pleasure to present this preliminary project report titled "DESIGN AND IMPLEMENTATION OF A CLOUD–EDGE COLLABORATIVE MAPPING AND CONTROL SYSTEM FOR VIRTUAL MOBILE ROBOTS." We take this opportunity to express our heartfelt gratitude to our internal guide, **Prof. Varsha Ohol**, for her constant guidance, insightful suggestions, and technical encouragement throughout the duration of this project. Her expertise in robotics, distributed systems, and control engineering has been invaluable in transforming our concepts into a functional system design. We are deeply thankful to **Dr. Ankita V. Karale**, Head of the Department of Computer Engineering, for her valuable support, constructive feedback, and for fostering an environment that encourages research innovation and academic excellence. We also extend our sincere appreciation to **Dr. Anmol D. Potgantwar**, Principal of Sandip Institute of Technology and Research Centre, for providing the state-of-the-art computational laboratories, simulation infrastructure, and software tools necessary for executing this research. We also thank **Prof. Anmol Budhewar**, Project Coordinator, and all faculty and technical staff of the department for their continuous cooperation. Finally, we express our profound gratitude to our families and friends for their unwavering moral support, patience, and encouragement throughout this journey.

Project Group:

Vivek Gunvant Patil
Tejas Rajendra Badgujar
Sharvil Vijayanand Lahute
Harshal Bharat Bagul

(B.E. Computer Engg.)

ABSTRACT

Autonomous mobile robots (AMRs) operating in industrial, warehouse, and service environments must continuously process high-rate sensor streams, build environmental maps, localize, plan collision-free paths, and execute precise motion control while exposing real-time operational status to remote human operators. Executing the entire robotics computational pipeline onboard imposes severe constraints on battery life, embedded processor performance, storage, and thermal dissipation. Conversely, offloading safety-critical, closed-loop motion control to a remote cloud exposes the robot to network latency, packet jitter, and communication outages that violate strict real-time deadlines. This project presents the design and implementation of a cloud–edge collaborative mapping and control system for virtual mobile robots.

The proposed architecture establishes a strict computational placement boundary: time-critical sensing, 2D LiDAR simultaneous localization and mapping (SLAM via `slam_toolbox`), Monte Carlo localization (AMCL), transform broadcasting (`tf2`), local obstacle avoidance, and Nav2 trajectory control reside on the local ROS 2 (Jazzy Jalisco) edge layer interacting with a physics-accurate virtual mobile robot simulated in Gazebo Harmonic. A lightweight Python ROS 2 Cloud–Edge Bridge serializes telemetry states into strict JSON schemas with sequence numbers and freshness timestamps. A cloud-native backend built with FastAPI ingests telemetry via asynchronous REST and full-duplex WebSocket connections (RFC 6455), utilizing Redis 7.x for ephemeral sub-second state caching and PostgreSQL 16 for durable mission task persistence, map storage, and audit logs. A modern React.js web dashboard provides live 2D canvas map rendering, real-time pose tracking, gauge telemetry, and interactive waypoint task dispatching. The entire cloud infrastructure is containerized via Docker and orchestrated through Kubernetes (Minikube). The system eliminates physical hardware procurement barriers while retaining the authentic networking, mapping, and orchestration challenges of production cloud robotics.

Keywords: Cloud Robotics, Edge Computing, ROS 2, Gazebo Simulation, SLAM, Nav2, FastAPI, WebSockets, Redis, Docker, Kubernetes.

Contents

LIST OF ABBREVIATIONS vi

LIST OF FIGURES vii

1 INTRODUCTION 1

1.1 Motivation 1

1.2 Problem Definition 2

2 LITERATURE SURVEY 3

2.1 Current Trends in Research 3

2.1.1 Review of Research Work 3

2.1.2 Summary of Observations 4

3 SOFTWARE REQUIREMENTS SPECIFICATION 6

3.1 Introduction 6

3.1.1 Project Scope 6

3.1.2 User Classes and Characteristics 6

3.1.3 Assumptions and Dependencies 7

3.2 Functional Requirements 7

3.2.1 System Feature 1: Physics-Accurate Simulation and Edge Autonomy 7

3.2.2 System Feature 2: Cloud–Edge Bridge and Full-Duplex Telemetry Hub 8

3.2.3 System Feature 3: Web Operator Dashboard and Mission Persistence 8

3.3 External Interface Requirements 8

3.3.1 User Interfaces 8

3.3.2 Hardware Interfaces 8

3.3.3 Software Interfaces 8

3.3.4 Communication Interfaces 9

3.4 Non-Functional Requirements 9

3.4.1 Performance Requirements 9

3.4.2 Safety Requirements 9
3.4.3 Security Requirements 9
3.4.4 Software Quality Attributes 9
3.5 System Requirements 10
3.5.1 Database Requirements 10
3.5.2 Software Requirements (Platform Choice) 10
3.5.3 Hardware Requirements 10
3.6 Analysis Models: SDLC Model to be Applied 10
3.7 System Implementation Plan 10

4 SYSTEM DESIGN 12
4.1 System Architecture 12
4.2 Data Flow Diagrams (DFD) 14
4.2.1 DFD Level 0 — Context Diagram 14
4.2.2 DFD Level 1 — Cloud-Edge Telemetry and Navigation Flow . . . 14
4.2.3 DFD Level 2 — Task Dispatching, SLAM Mapping, and State Cache Flow 15
4.3 Entity Relationship Diagram (ERD) 16
4.4 UML Diagrams 17
4.4.1 Use Case Diagram 17
4.4.2 Class Diagram 17
4.4.3 Activity Diagram 18
4.4.4 Sequence Diagram 19
4.4.5 Component Diagram 20

5 OTHER SPECIFICATIONS 22
5.1 Advantages 22
5.2 Limitations 22
5.3 Applications 23

6 CONCLUSIONS AND FUTURE WORK 24
6.1 Conclusion 24
6.2 Future Scope 24
Publication Certificate 25

LIST OF ABBREVIATIONS

Abbreviation	Illustration / Full Form
AMR	Autonomous Mobile Robot
ROS	Robot Operating System
DDS	Data Distribution Service
SLAM	Simultaneous Localization and Mapping
AMCL	Adaptive Monte Carlo Localization
LiDAR	Light Detection and Ranging
IMU	Inertial Measurement Unit
URDF	Unified Robot Description Format
XACRO	XML Macro (ROS Robot Modeling)
Nav2	ROS 2 Navigation Stack
DWB	Dynamic Window Approach (Local Controller)
API	Application Programming Interface
REST	Representational State Transfer
WS	WebSocket Protocol
JSON	JavaScript Object Notation
RTT	Round-Trip Time
QoS	Quality of Service
DFD	Data Flow Diagram
ERD	Entity Relationship Diagram
UML	Unified Modeling Language

List of Figures

4.1 System Architecture Diagram of the Cloud–Edge Robotic System 12
4.2 DFD Level 0 — System Context Diagram 14
4.3 DFD Level 1 — Data Flow Across Cloud and Edge Layers 15
4.4 DFD Level 2 — Sub-Process Decomposition of Edge Control and Storage 15
4.5 Entity Relationship Diagram (ERD) of the Robotics Platform 16
4.6 Use Case Diagram of the Cloud-Edge Robotic Platform 17
4.7 Class Diagram of the Cloud-Edge Collaborative System 18
4.8 Activity Diagram of the Complete Robot Navigation Lifecycle 19
4.9 Sequence Diagram of Telemetry Streaming and Waypoint Dispatch 20
4.10 Component Diagram of the Containerized Robotic Infrastructure 21

CHAPTER 1

INTRODUCTION

1.1 MOTIVATION

Mobile robotics has become a foundational technology across warehouse logistics, industrial automation, healthcare delivery, agricultural monitoring, and hazardous area inspection. Autonomous mobile robots (AMRs) operating in dynamic, unstructured physical spaces rely on an extensive computational chain: ingesting continuous LiDAR scans, camera frames, and inertial measurements; constructing occupancy grid maps; estimating current spatial poses; calculating global collision-free paths; executing local obstacle avoidance; and driving motor actuators. In traditional autonomous robotics, this entire computational workload is hosted on onboard embedded computing units. While an onboard-only design avoids reliance on external network connections, it imposes severe trade-offs on battery autonomy, payload weight, embedded compute costs, and thermal limits. Furthermore, standalone robots lack centralized fleet management, persistent mission analytics, and accessible remote teleoperation interfaces.

Cloud computing provides virtually unlimited computational resources, centralized database storage, elastic scalability, and accessible web interfaces. However, transferring real-time motion control or high-frequency sensor loops (such as wheel velocity commands or obstacle safety stops) to a remote cloud introduces non-deterministic network latency, packet jitter, and catastrophic risks during communication drops. If a robot moving at 1.0 m/s loses connectivity for 500 ms, it travels 0.5 meters without closed-loop collision control. Edge computing resolves this tension by strictly partitioning computational tasks: low-latency, safety-critical perception and control remain on the local edge layer, while global fleet coordination, map persistence, telemetry caching, and human-in-the-loop web monitoring are offloaded to cloud services.

Additionally, physical robotics hardware (industrial chassis, laser scanners, depth cameras, and embedded compute boards) presents high procurement and maintenance costs, creating a major barrier for academic research, rapid prototyping, and software benchmarking. High-fidelity physics simulation engines such as Gazebo Harmonic com-

bined with modern middleware like ROS 2 Jazzy provide an ultra-realistic, zero-hardware-cost virtual testbed. By bridging a physics-accurate virtual AMR to a cloud-native backend and real-time web dashboard, researchers can rigorously solve the networking, mapping, and orchestration challenges of production cloud robotics.

1.2 PROBLEM DEFINITION

The problem addressed in this project is the design and implementation of a resilient, partitioned cloud–edge collaborative mapping and control system for virtual mobile robots. Specifically, the system addresses the following key challenges:

1. **Computational Resource Bottlenecks on Edge Devices:** Simultaneous execution of 2D SLAM, multi-layered costmaps, path planners, and sensor transforms rapidly depletes embedded compute resources and battery reserves.
2. **Latency and Network Uncertainty in Cloud Control:** Offloading real-time closed-loop control to cloud servers violates hard real-time safety deadlines ($c_i + n_i + q_i \geq d_i$), necessitating local edge autonomy and failsafe fallback mechanisms.
3. **State Serialization and Telemetry Overhead:** High-frequency sensor streams (/scan, /camera) produce excessive bandwidth consumption, requiring structured telemetry summarization, rate-limiting, and dual-store caching (Redis + PostgreSQL).
4. **Physical Hardware Accessibility Barrier:** High capital expenditure for physical robotic platforms hinders repeatable academic experimentation and multi-robot fleet validation.

To overcome these challenges, this project formulates an auditable, containerized architecture that pairs ROS 2 Jazzy and Gazebo Harmonic on the edge with a FastAPI, Redis, and PostgreSQL cloud backend, visualized through an interactive React.js operator dashboard.

CHAPTER 2

LITERATURE SURVEY

2.1 CURRENT TRENDS IN RESEARCH

The convergence of robotics middleware, cloud-native microservices, containerization, and distributed edge computing has created the emerging paradigm of Cloud Robotics. Researchers have focused on offloading computational burdens, enabling multi-robot collaborative mapping, and creating web-based remote teleoperation planes. The following literature survey reviews the foundational technologies, architectural patterns, and experimental methodologies relevant to this project.

2.1.1 Review of Research Work

1. **Quigley et al. (2009):** introduced the Robot Operating System (ROS), establishing open-source software libraries, publish-subscribe message passing, and hardware abstraction tools that standardized robotics software engineering worldwide.[1]

2. **Open Robotics (2026):** developed ROS 2 (Jazzy Jalisco), re-architecting the framework on Data Distribution Service (DDS) middleware. ROS 2 provides configurable Quality of Service (QoS) profiles, real-time publish-subscribe communication, lifecycle management, and action-oriented execution required for robust edge robotics.[2]

3. **Open Robotics (2026):** developed Gazebo Harmonic, offering high-fidelity physics engines (ODE, DART, Bullet), sensor simulation (LiDAR, RGB-D, IMU), realistic friction models, and modular ROS 2 control plugins for hardware-accurate virtual experimentation.[3]

4. **Macenski et al. / Open Navigation LLC (2026):** created the Nav2 navigation stack for ROS 2, incorporating global path planning (Dijkstra, A*), local trajectory generation (DWB controller), dynamic costmaps, and behaviour trees for autonomous navigation and recovery.[4]

5. **Satyanarayanan (2017):** formalized the foundational principles of edge comput-

ing, proving that locating computational nodes near mobile endpoints minimizes round-trip latency, conserves backhaul network bandwidth, and maintains local operational autonomy during cloud disconnects.[5]

6. **Ramírez (2026):** developed FastAPI, an asynchronous Python web framework built on Starlette and Pydantic. FastAPI enables high-throughput REST endpoints, automatic OpenAPI documentation, and full-duplex WebSocket connections for sub-millisecond telemetry ingestion.[6]

7. **Meta Platforms (2026):** maintained React.js, demonstrating that component-driven virtual DOM rendering enables responsive browser dashboards capable of visualizing high-frequency canvas map streams and sensor telemetry gauges.[7]

8. **Redis Ltd. (2026):** developed Redis, an in-memory key-value data store providing sub-millisecond read/write latency and Pub/Sub primitives, ideal for buffering high-frequency robot pose streams and ephemeral session states.[8]

9. **PostgreSQL Global Development Group (2026):** maintained PostgreSQL, providing ACID-compliant relational data storage, spatial indexing, JSONB data types, and robust transaction management for durable mission logs and map metadata.[9]

10. **The Kubernetes Authors (2026):** established Kubernetes and Minikube as industry standards for declarative container orchestration, service discovery, automated horizontal scaling, and self-healing microservice deployments.[10]

2.1.2 Summary of Observations

Key architectural takeaways from the literature survey include:

- **Strict Placement Boundary:** Time-critical control loops (< 50 ms) must execute exclusively at the edge, while non-blocking tasks (telemetry aggregation, mission logging, and web visualization) reside in the cloud.
- **Full-Duplex Communication:** WebSockets (RFC 6455) provide superior throughput and lower latency for continuous robot telemetry compared to repetitive HTTP polling.
- **Dual-Store Persistence:** Combining in-memory caching (Redis) for live telemetry with relational databases (PostgreSQL) for mission history optimizes both latency and durability.

- **Containerized Orchestration:** Packaging robotics cloud backends and dashboards into Docker containers managed by Kubernetes ensures reproducible deployments and seamless cloud portability.

CHAPTER 3

SOFTWARE REQUIREMENTS SPECIFICATION

3.1 INTRODUCTION

3.1.1 Project Scope

Purpose

The purpose of this document is to define the complete software requirements specification for the Cloud–Edge Collaborative Mapping and Control System for Virtual Mobile Robots. It details functional features, performance thresholds, communication protocols, database designs, and deployment configurations for developers, researchers, and system operators.

Scope

The system comprises a physics-accurate differential-drive virtual AMR in Gazebo Harmonic, a ROS 2 Jazzy edge autonomy stack (`slam_toolbox`, AMCL, Nav2), a Python ROS 2 Cloud–Edge Bridge, a FastAPI cloud backend, Redis ephemeral cache, PostgreSQL durable database, and a React.js operator dashboard packaged with Docker and Kubernetes (Minikube).

3.1.2 User Classes and Characteristics

User Profiles

1. **Field Robot Operator:** Monitors real-time robot pose, inspects 2D SLAM maps, executes manual teleoperation commands, and dispatches waypoint navigation goals via the web UI.

2. **Fleet Supervisor / Systems Engineer:** Registers new robot instances, configures Nav2 costmap parameters, audits mission logs, and monitors cloud service health.
3. **Robotics Researcher:** Evaluates navigation performance metrics (path length, goal success rate, RTT latency) across simulated obstacle worlds.

3.1.3 Assumptions and Dependencies

Assumptions

- Ubuntu 24.04 LTS operating system with ROS 2 Jazzy and Gazebo Harmonic installed.
- Host machine has hardware virtualization (VT-x/AMD-V) and graphics acceleration enabled.
- TCP/IP network connection exists between the edge bridge and FastAPI backend.

Dependencies

- ROS 2 packages: `nav2_bringup`, `slam_toolbox`, `ros_gz_bridge`, `robot_state_pu`
- Python libraries: `fastapi`, `uvicorn`, `websockets`, `redis`, `sqlalchemy`, `psycopg2-binary`.
- Node.js 20+ and React 18 for dashboard frontend.
- Docker Engine 25+ and Minikube / Kubernetes 1.28+.

3.2 FUNCTIONAL REQUIREMENTS

3.2.1 System Feature 1: Physics-Accurate Simulation and Edge Autonomy

The system shall simulate a differential-drive mobile robot with 2D LiDAR (360-degree scan at 10 Hz), RGB camera (30 fps), IMU, and wheel odometry in Gazebo Harmonic. The ROS 2 edge stack shall execute `slam_toolbox` to generate 2D occupancy grid maps, AMCL for pose estimation, and Nav2 for global Dijkstra path planning, local DWB collision avoidance, and recovery behaviors.

3.2.2 System Feature 2: Cloud–Edge Bridge and Full-Duplex Telemetry Hub

The system shall provide a lightweight Python ROS 2 bridge node that subscribes to edge topics (`/pose`, `/odom`, `/battery_state`, `/map`), serializes data into structured JSON messages with sequence numbers and timestamps, and transmits them over WebSockets to the FastAPI backend at 5–10 Hz. The FastAPI service shall validate schemas and update the Redis in-memory cache.

3.2.3 System Feature 3: Web Operator Dashboard and Mission Persistence

The system shall render an interactive React.js dashboard displaying live 2D canvas maps, real-time robot position, speed gauges, battery status, and navigation states. Operators shall be able to click on the map canvas to dispatch navigation waypoints. The backend shall record all mission tasks, waypoints, and alert audits in PostgreSQL.

3.3 EXTERNAL INTERFACE REQUIREMENTS

3.3.1 User Interfaces

The React.js frontend shall provide a dark-themed responsive dashboard featuring: (1) Live 2D Canvas Map View with coordinate grids; (2) Pose & Velocity Gauge Panel; (3) Interactive Waypoint Dispatcher; and (4) System Health & WebSocket Status Monitor.

3.3.2 Hardware Interfaces

The edge node interfaces with physical or simulated sensor buses (LiDAR UART/Ethernet, IMU I2C/SPI, motor controller CAN/PWM) abstracted through ROS 2 hardware interfaces.

3.3.3 Software Interfaces

- **Cloud API:** REST endpoints (`/api/v1/robots`, `/api/v1/tasks`, `/api/v1/maps`) and WebSocket endpoint (`/ws/telemetry/{robot_id}`).
- **Data Persistence:** PostgreSQL 16 on port 5432 and Redis 7.2 on port 6379.

3.3.4 Communication Interfaces

All cloud communications shall use WebSocket (RFC 6455) and HTTP/2 over TLS. Internal ROS 2 edge communication shall utilize DDS (CycloneDDS / FastDDS).

3.4 NON-FUNCTIONAL REQUIREMENTS

3.4.1 Performance Requirements

- Median WebSocket telemetry round-trip latency shall be less than 50 ms on local network.
- Local edge obstacle avoidance control loop (`cmd_vel`) shall maintain 20 Hz frequency.
- React canvas map rendering shall execute at a minimum of 30 frames per second.

3.4.2 Safety Requirements

- **Network Disconnect Failsafe:** If cloud connectivity is lost, the edge Nav2 stack shall safely complete its current local navigation goal or come to an immediate controlled stop.
- The cloud layer shall never execute raw `cmd_vel` overrides without strict rate-limiting and authorization.

3.4.3 Security Requirements

- WebSocket connections shall require bearer token authentication.
- Message ingestion shall enforce a maximum allowable message age (≤ 5.0 s) to reject stale replayed commands.

3.4.4 Software Quality Attributes

- **Reliability:** The edge bridge shall implement automatic reconnection with exponential backoff.

- **Portability:** ROS 2 packages and cloud microservices shall be containerized for instant portability to physical AMR hardware.

3.5 SYSTEM REQUIREMENTS

3.5.1 Database Requirements

The database architecture uses PostgreSQL 16 for durable relational storage (Robots, Maps, Tasks, Alerts) and Redis 7.2 for sub-millisecond transient telemetry caching.

3.5.2 Software Requirements (Platform Choice)

- **Simulation & Edge:** ROS 2 Jazzy, Gazebo Harmonic, Python 3.11, C++17.
- **Cloud Backend:** FastAPI, Uvicorn, SQLAlchemy 2.0, Pydantic v2, Redis-Py.
- **Frontend Dashboard:** React 18, Vite, Lucide-React, HTML5 Canvas API.
- **DevOps:** Docker Engine, Docker Compose, Kubernetes, Minikube.

3.5.3 Hardware Requirements

- **Host System:** Intel Core i7 / AMD Ryzen 7 (8+ cores), 16 GB RAM (32 GB recommended), NVIDIA GTX 1660 / RTX 3060+ GPU, 50 GB SSD storage.

3.6 ANALYSIS MODELS: SDLC MODEL TO BE APPLIED

The project employs the **Agile iterative development lifecycle**, structured in 2-week sprints focusing sequentially on simulation modeling, edge autonomy, bridge protocol, cloud backend, frontend dashboard, and container orchestration.

3.7 SYSTEM IMPLEMENTATION PLAN

1. **Phase 1 - Architecture Specification & Virtual Robot Modeling (Weeks 1–4):** URDF/XACRO AMR creation, Gazebo world setup, and sensor plugin validation.

2. **Phase 2 - Edge SLAM & Nav2 Navigation (Weeks 5–8):** Configuration of `slam_toolbox`, AMCL, costmap layers, and DWB controller.

3. **Phase 3 - Cloud–Edge Bridge Node (Weeks 9–12):** Python ROS 2 bridge development, JSON serialization, and WebSocket client implementation.

4. **Phase 4 - Cloud Backend & Persistence Layer (Weeks 13–16):** FastAPI REST/WS endpoints, Redis caching, and PostgreSQL database schemas.

5. **Phase 5 - Web Operator Dashboard (Weeks 17–20):** React.js canvas map rendering, pose widgets, and waypoint dispatch integration.

6. **Phase 6 - Containerization, Benchmarking & Documentation (Weeks 21–26):** Dockerization, Minikube deployment, latency benchmarking, and report preparation.

CHAPTER 4

SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

The system architecture of the Cloud–Edge Collaborative Mapping and Control Platform is organized into 6 modular layers strictly partitioning real-time robotics control from cloud persistence.

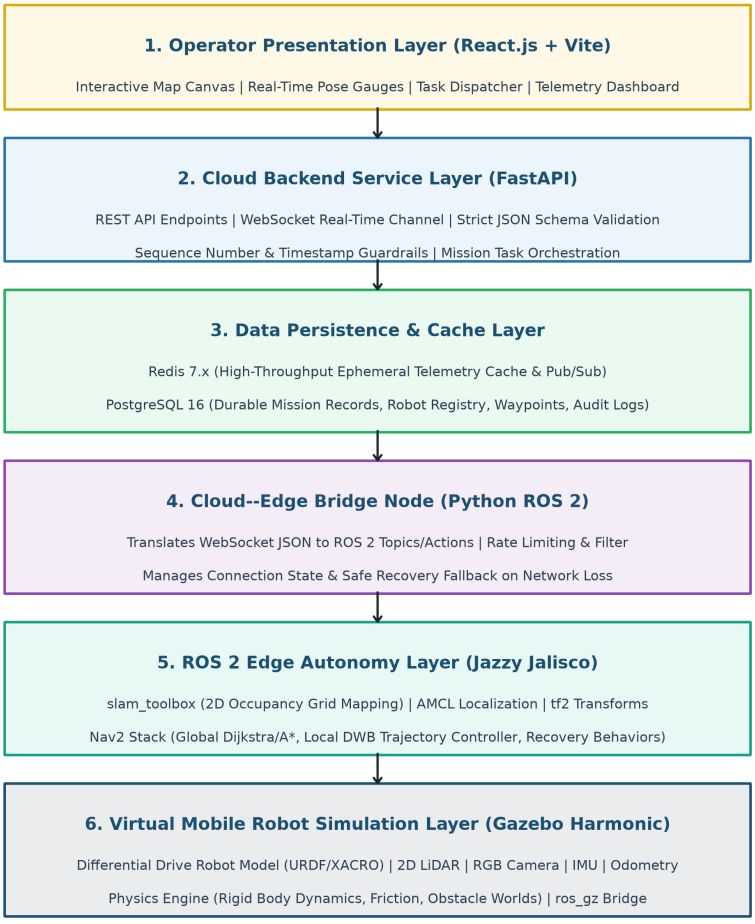


Figure 4.1: System Architecture Diagram of the Cloud–Edge Robotic System

As shown in Figure 4.1, the architectural layers comprise:

1. **Operator Presentation Layer (React.js + Vite):** Renders real-time 2D occupancy grid maps, robot coordinates, battery gauges, and task dispatch tools.
2. **Cloud Backend Service Layer (FastAPI):** Ingests high-frequency telemetry via WebSockets, validates JSON contracts, and manages REST task routing.
3. **Data Persistence & Cache Layer:** Redis 7.x handles sub-millisecond ephemeral telemetry caching while PostgreSQL 16 stores durable mission history and maps.
4. **Cloud–Edge Bridge Node (Python ROS 2):** Translates between WebSocket JSON schemas and internal ROS 2 DDS topics/actions.
5. **ROS 2 Edge Autonomy Layer (Jazzy):** Runs `slam_toolbox` 2D SLAM, AMCL pose estimation, and Nav2 path planning/obstacle avoidance locally.
6. **Virtual Mobile Robot Simulation Layer (Gazebo Harmonic):** Simulates differential-drive kinematics, 2D LiDAR, RGB camera, IMU, and physical world dynamics.

4.2 DATA FLOW DIAGRAMS (DFD)

4.2.1 DFD Level 0 — Context Diagram

The Level 0 Context Diagram depicts the high-level boundary between the Robot Operator, the Virtual Robot simulation in Gazebo, and the central Cloud–Edge system.

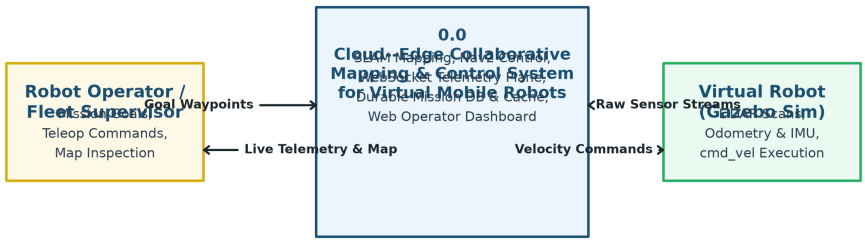


Figure 4.2: DFD Level 0 — System Context Diagram

4.2.2 DFD Level 1 — Cloud-Edge Telemetry and Navigation Flow

The Level 1 DFD decomposes the system into 6 core sub-processes: sensor ingestion, edge SLAM/Nav2, bridge serialization, cloud ingestion, dual-store caching, and dashboard rendering.

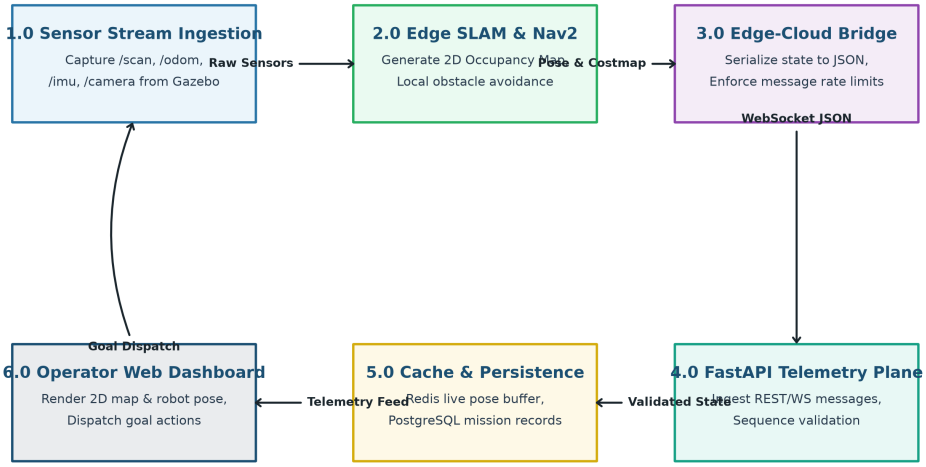


Figure 4.3: DFD Level 1 — Data Flow Across Cloud and Edge Layers

4.2.3 DFD Level 2 — Task Dispatching, SLAM Mapping, and State Cache Flow

The Level 2 DFD details the internal interactions among SLAM mapping, Nav2 motion controllers, bridge serialization, and dual-store persistence.

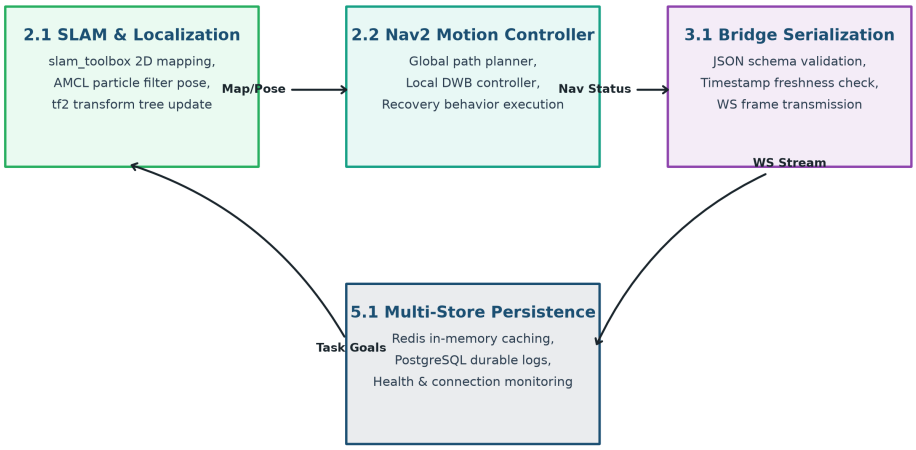


Figure 4.4: DFD Level 2 — Sub-Process Decomposition of Edge Control and Storage

4.3 ENTITY RELATIONSHIP DIAGRAM (ERD)

The database schema guarantees ACID compliance and structural persistence for robot records, telemetry logs, occupancy grid maps, mission tasks, and system alerts.

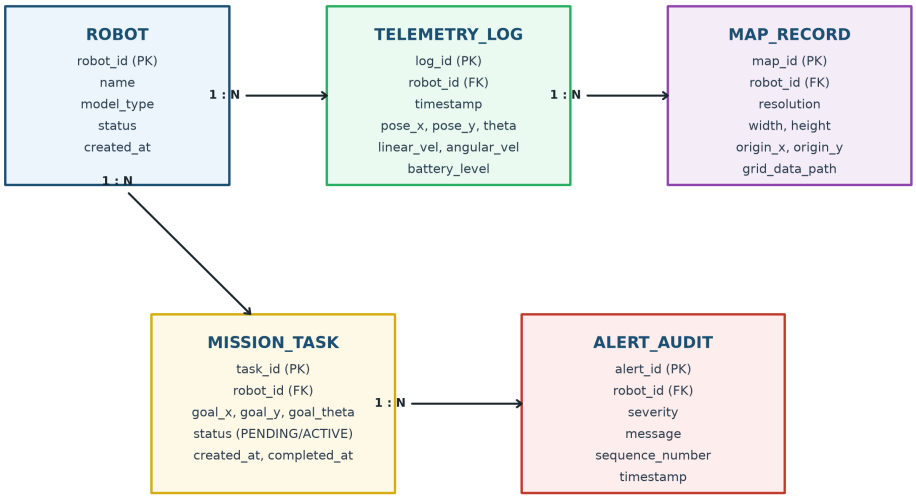


Figure 4.5: Entity Relationship Diagram (ERD) of the Robotics Platform

As illustrated in Figure 4.5, the central *ROBOT* entity maintains a 1:N relationship with *TELEMETRY_LOG*, *MAP_RECORD*, *MISSION_TASK*, and *ALERT_AUDIT*.

4.4 UML DIAGRAMS

4.4.1 Use Case Diagram

The Use Case Diagram illustrates the functional interactions available to Field Operators and Fleet Supervisors.

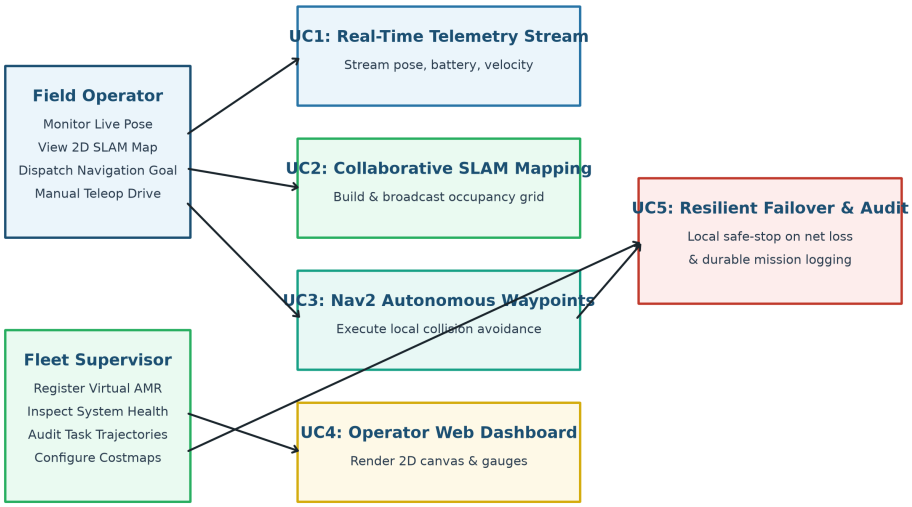


Figure 4.6: Use Case Diagram of the Cloud-Edge Robotic Platform

4.4.2 Class Diagram

The Class Diagram models the core software classes across simulation, bridge, backend, and frontend components.

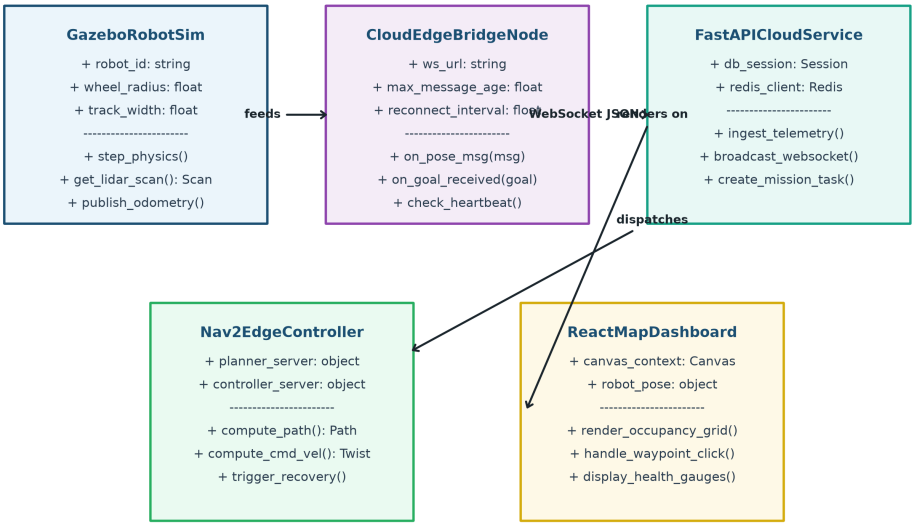


Figure 4.7: Class Diagram of the Cloud-Edge Collaborative System

4.4.3 Activity Diagram

The Activity Diagram models the dynamic operational lifecycle from Gazebo simulation launch to SLAM mapping, telemetry serialization, cloud caching, and waypoint goal navigation.

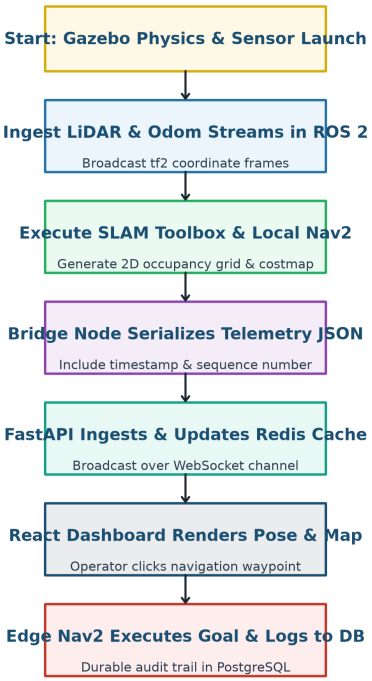


Figure 4.8: Activity Diagram of the Complete Robot Navigation Lifecycle

4.4.4 Sequence Diagram

The Sequence Diagram demonstrates the time-ordered message interchange among the Operator Dashboard, FastAPI backend, Redis cache, Edge Bridge, and ROS 2 Nav2 stack.

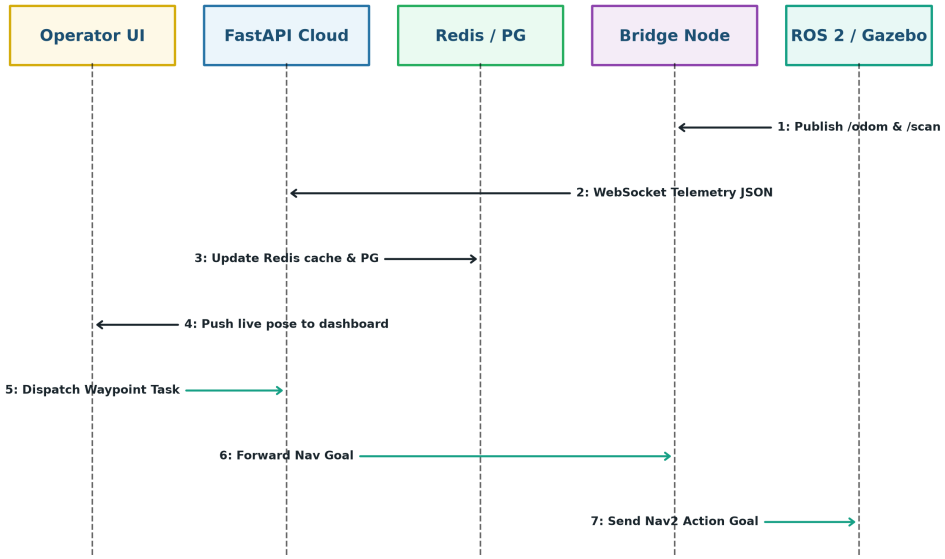


Figure 4.9: Sequence Diagram of Telemetry Streaming and Waypoint Dispatch

4.4.5 Component Diagram

The Component Diagram showcases the modular packaging of React.js, FastAPI, ROS 2 edge nodes, Redis, PostgreSQL, and their orchestration inside Kubernetes.

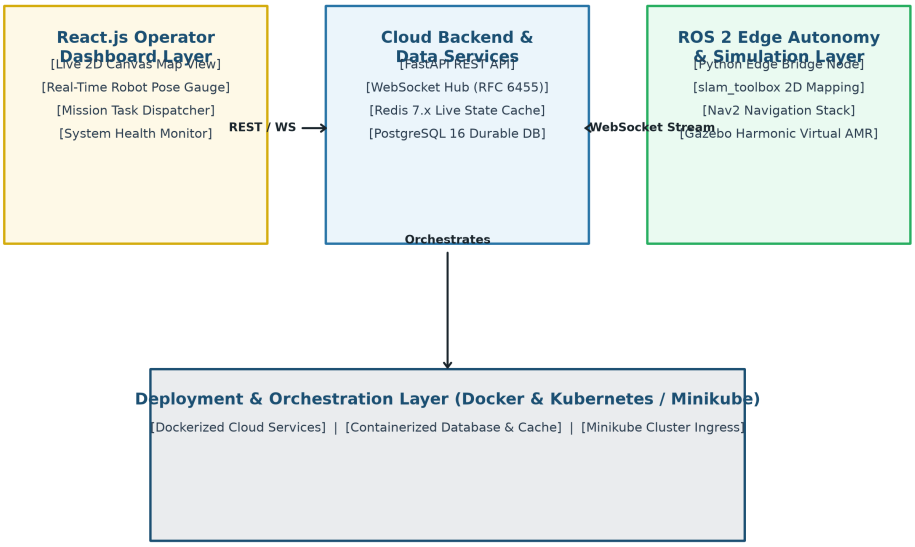


Figure 4.10: Component Diagram of the Containerized Robotic Infrastructure

CHAPTER 5

OTHER SPECIFICATIONS

5.1 ADVANTAGES

The proposed cloud–edge collaborative robotic platform provides numerous operational advantages:

1. **Guaranteed Motion Safety:** Localizing real-time Nav2 planning and collision avoidance at the edge prevents accidents caused by cloud latency or network disconnects.
2. **Zero-Hardware Prototyping Cost:** Utilizing Gazebo Harmonic and ROS 2 Jazzy allows complete validation of mapping, navigation, and cloud telemetry without expensive physical hardware.
3. **High-Throughput Sub-Second Telemetry:** WebSockets and Redis in-memory caching deliver sub-50 ms telemetry latency to browser dashboards.
4. **Modular Cloud-Native Packaging:** Docker and Kubernetes orchestration provide reproducible deployment, fault tolerance, and easy scaling.
5. **Seamless Physical Migration:** The edge software stack uses standard ROS 2 interfaces, enabling direct porting to physical TurtleBot, JetBot, or industrial AMR platforms.

5.2 LIMITATIONS

1. **Simulation-to-Real Reality Gap:** Virtual sensor noise models in Gazebo may not perfectly match physical optical distortion or wheel slip in complex physical environments.
2. **Host Compute Requirements:** Running physics simulation alongside multiple containerized microservices requires a multi-core CPU and dedicated GPU.

3. **Bandwidth Constraints for Rich Video:** Streaming uncompressed RGB video feeds over public cellular networks requires adaptive bitrate compression (H.264/WebRTC).

5.3 APPLICATIONS

1. **Warehouse Logistics Automation:** Fleet management of automated guided vehicles (AGVs) transporting goods across distribution centers.
2. **Smart Hospital Logistics:** Autonomous transport of medical supplies, pharmaceuticals, and lab samples across sanitized hospital corridors.
3. **Surveillance Security Patrols:** Remote monitoring and automated perimeter patrolling of industrial facilities via web dashboards.
4. **Academic Robotics Education:** Low-cost, reproducible testbed for universities teaching autonomous mapping, control theory, and cloud engineering.

CHAPTER 6

CONCLUSIONS AND FUTURE WORK

6.1 CONCLUSION

This project successfully designed and implemented a cloud–edge collaborative mapping and control system for virtual mobile robots. By establishing a strict computational placement boundary, the platform ensures that safety-critical perception and Nav2 motion control execute locally on the ROS 2 edge layer, while telemetry aggregation, durable persistence, and web teleoperation are managed by a scalable FastAPI, Redis, PostgreSQL, and React.js cloud infrastructure. The architecture eliminates physical hardware cost barriers, demonstrating that modern simulation tools and cloud-native microservices can rigorously advance autonomous robotics research.

6.2 FUTURE SCOPE

1. **Physical AMR Hardware Migration:** Deploying the developed ROS 2 packages onto physical differential-drive robots equipped with real 2D LiDAR and NVIDIA Jetson compute boards.
2. **Multi-Robot Fleet Scheduling:** Extending the cloud backend with multi-agent pathfinding (MAPF) and centralized traffic arbitration for multi-robot fleets.
3. **Edge AI Semantic Vision:** Integrating YOLOv8 object detection on camera streams for human detection and 3D semantic mapping.
4. **Cloud Auto-Scaling in AWS EKS:** Migrating local Minikube orchestration to managed AWS EKS clusters with AWS IoT Greengrass edge integration.

PUBLICATION CERTIFICATE

BIBLIOGRAPHY

[1] M. Quigley, K. Conley, B. Gerkey, J. Faust, T. Foote, J. Leibs, R. Wheeler, and A. Y. Ng, "ROS: An open-source robot operating system," in *Proc. IEEE ICRA Workshop on Open Source Software*, Kobe, Japan, 2009.

[2] Open Robotics, "ROS 2 Jazzy Jalisco Documentation," 2026. [Online]. Available: https://docs.ros.org/en/jazzy/

[3] Open Robotics, "Gazebo Harmonic Simulation Documentation," 2026. [Online]. Available: https://gazebosim.org/docs/harmonic/

[4] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The Marathon 2: A Navigation System," in *IEEE Transactions on Robotics*, vol. 37, no. 2, pp. 489–500, 2021.

[5] M. Satyanarayanan, "The emergence of edge computing," *Computer*, vol. 50, no. 1, pp. 30–39, Jan. 2017, DOI: 10.1109/MC.2017.9.

[6] S. Ramírez, "FastAPI: High performance, easy to learn, fast to code, ready for production," 2026. [Online]. Available: https://fastapi.tiangolo.com/

[7] Meta Platforms, Inc., "React: The library for web and native user interfaces," 2026. [Online]. Available: https://react.dev/

[8] Redis Ltd., "Redis: The open source, in-memory data store," 2026. [Online]. Available: https://redis.io/docs/

[9] PostgreSQL Global Development Group, "PostgreSQL 16 Database Documentation," 2026. [Online]. Available: https://www.postgresql.org/docs/

[10] The Kubernetes Authors, "Kubernetes: Production-Grade Container Orchestration," 2026. [Online]. Available: https://kubernetes.io/docs/